

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 7**



TREE (POHON)

Oleh:

Muhammad Rafly Ash-Shadiq

NIM. 2510817310007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 7

Laporan Praktikum Algoritma & Struktur Data Modul 7: Tree (Pohon) ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rafly Ash-Shadiq
NIM : 2510817310007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
SOAL 1	1
A. Source Code	3
B. Output Program	6
C. Pembahasan	10
LINK GITHUB	17

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Tambah Untuk Input Data	6
Gambar 2 Screenshot Output Program Tambah Untuk Input Data	7
Gambar 3 Screenshot Output Program Tambah Untuk Input Data	7
Gambar 4 Screenshot Output Program Tambah Untuk Input Data	7
Gambar 5 Screenshot Output Program Tambah Untuk Input Data	8
Gambar 6 Screenshot Output Program Tambah Input Data Yang Sama.....	8
Gambar 7 Screenshot Output Program PreOrder.....	8
Gambar 8 Screenshot Output Program PreOrder Data Kosong.....	9
Gambar 9 Screenshot Output Program InOrder.....	9
Gambar 10 Screenshot Output Program InOrder Data Kosong.....	9
Gambar 11 Screenshot Output Program PostOrder	10
Gambar 12 Screenshot Output Program PostOrder Data Kosong	10
Gambar 13 Screenshot Output Program Exit.....	10

DAFTAR TABEL

Tabel 1 Source Code Jawaban Untuk Soal Nomor 1	3
--	---

SOAL 1

Cobalah program berikut, perbaiki output, lengkapi fungsi inOrder dan postOrder pada coding, running, simpan program !

```
1  #include <stdio.h>
2  #include <conio.h>
3  #include <stdlib.h>
4  #include <iostream>
5
6  using namespace std;
7  struct Node
8  {
9      int data;
10     Node *kiri;
11     Node *kanan;
12 };
13
14 void tambah(Node **root, int databaru)
15 {
16     if (*root == NULL)
17     {
18         Node *baru;
19         baru = new Node;
20         baru->data = databaru;
21         baru->kiri = NULL;
22         baru->kanan = NULL;
23         (*root) = baru;
24         (*root)->kiri = NULL;
25         (*root)->kanan = NULL;
26         cout << "Data bertambah";
27     }
28     else if (databaru < (*root)->data)
29         tambah(&(*root)->kiri, databaru);
30     else if (databaru > (*root)->data)
31         tambah(&(*root)->kanan, databaru);
32     else if (databaru == (*root)->data)
33         cout << "Data sudah ada";
34 }
35
36 void preOrder(Node *root)
37 {
38     if (root != NULL)
39     {
40         cout << root->data;
41         preOrder(root->kiri);
42         preOrder(root->kanan);
43     }
44 }
45
46 void inOrder(Node *root)
47 {
48     if (root != NULL)
```

```

49
50
51
52
53
54 }
55
56 void postOrder(Node *root)
57 {
58
59
60
61
62
63
64 }
65
66 int main()
67 {
68     int pil, data;
69     Node *pohon;
70     pohon = NULL;
71     do
72     {
73         system("cls");
74         cout << "1. Tambah\n";
75         cout << "2. PreOrder\n";
76         cout << "3. inOrder\n";
77         cout << "4. PostOrder\n";
78         cout << "5. Exit\n";
79         cout << "\nPilihan : ";
80         cin >> pil;
81         switch (pil)
82         {
83             case 1:
84                 cout << "\n INPUT : ";
85                 cout << "\n -----";
86                 cout << "\n Data baru : ";
87                 cin >> data;
88                 tambah(&pohon, data);
89                 break;
90             case 2:
91                 cout << "PreOrder";
92                 cout << "\n-----\n";
93                 if (pohon != NULL)
94                 {
95                     preOrder(pohon);
96

```

```

97         else
98             cout << "Masih Kosong";
99         break;
100     case 3:
101         cout << "InOrder";
102         cout << "\n-----\n";
103         if (pohon != NULL)
104         {
105             inOrder(pohon);
106         }
107         else
108             cout << "Masih Kosong";
109         break;
110     case 4:
111         cout << "PostOrder";
112         cout << "\n-----\n";
113         if (pohon != NULL)
114         {
115             postOrder(pohon);
116         }
117         else
118             cout << "Masih Kosong";
119         break;
120     case 5:
121         return 0;
122     }
123     _getch();
124 } while (pil != 5);
125 return EXIT_FAILURE;
126 }

```

A. Source Code

Tabel 1 Source Code Jawaban Untuk Soal Nomor 1

1.	c	#include <stdio.h>
2.		#include <conio.h>
3.		#include <stdlib.h>
4.		#include <iostream>
5.		
6.		using namespace std;
7.		
8.		struct Node {
9.		int data;
10.		Node *kiri;
11.		Node *kanan;
12.		};
13.		


```

14. void tambah(Node **root, int databaru) {
15.     if (*root == NULL) {
16.         Node *baru;
17.         baru = new Node;
18.         baru->data = databaru;
19.         baru->kiri = NULL;
20.         baru->kanan = NULL;
21.         (*root) = baru;
22.         (*root)->kiri = NULL;
23.         (*root)->kanan = NULL;
24.         cout << "Data bertambah";
25.     }
26.
27.     else if (databaru < (*root)->data)
28.         tambah(&(*root)->kiri, databaru);
29.     else if (databaru > (*root)->data)
30.         tambah(&(*root)->kanan, databaru);
31.     else if (databaru == (*root)->data)
32.         cout << "Data sudah ada";
33. }
34.
35. void preOrder(Node *root) {
36.     if (root != NULL) {
37.         cout << root->data << " ";
38.         preOrder(root->kiri);
39.         preOrder(root->kanan);
40.     }
41. }
42.
43. void inOrder(Node *root) {
44.     if (root != NULL) {
45.         inOrder(root->kiri);
46.         cout << root->data << " ";
47.         inOrder(root->kanan);
48.     }
49. }
50.
51. void postOrder(Node *root) {
52.     if (root != NULL) {
53.         postOrder(root->kiri);

```

```

54.         postOrder(root->kanan);
55.         cout << root->data << " ";
56.     }
57. }
58.
59. int main() {
60.     int pil, data;
61.     Node *pohon;
62.     pohon = NULL;
63.
64.     do {
65.         system("cls");
66.         cout << "1. Tambah\n";
67.         cout << "2. PreOrder\n";
68.         cout << "3. inOrder\n";
69.         cout << "4. PostOrder\n";
70.         cout << "5. Exit\n";
71.         cout << "\nPilihan : ";
72.         cin >> pil;
73.
74.         switch (pil) {
75.             case 1:
76.                 cout << "\n INPUT : ";
77.                 cout << "\n -----";
78.                 cout << "\n Data baru : ";
79.                 cin >> data;
80.                 tambah(&pohon, data);
81.                 break;
82.             case 2:
83.                 cout << "PreOrder";
84.                 cout << "\n-----\n";
85.                 if (pohon != NULL) {
86.                     preOrder(pohon);
87.                 }
88.                 else
89.                     cout << "Masih Kosong";
90.                 break;
91.             case 3:
92.                 cout << "InOrder";
93.                 cout << "\n-----\n";

```

```

94.         if (pohon != NULL) {
95.             inOrder(pohon);
96.         }
97.         else
98.             cout << "Masih Kosong";
99.         break;
100.     case 4:
101.         cout << "PostOrder";
102.         cout << "\n-----\n";
103.         if (pohon != NULL) {
104.             postOrder(pohon);
105.         }
106.         else
107.             cout << "Masih Kosong";
108.         break;
109.     case 5:
110.         return 0;
111.     }
112.     _getch();
113. }
114.
115. while (pil != 5);
116. return EXIT_FAILURE;
117. }

```

B. Output Program

```

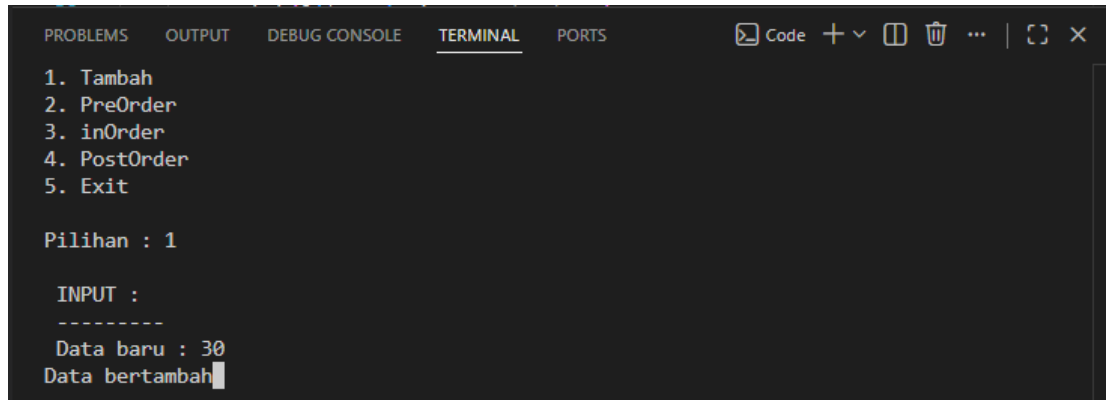
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 50
Data bertambah

```

Gambar 1 Screenshot Output Program Tambah Untuk Input Data

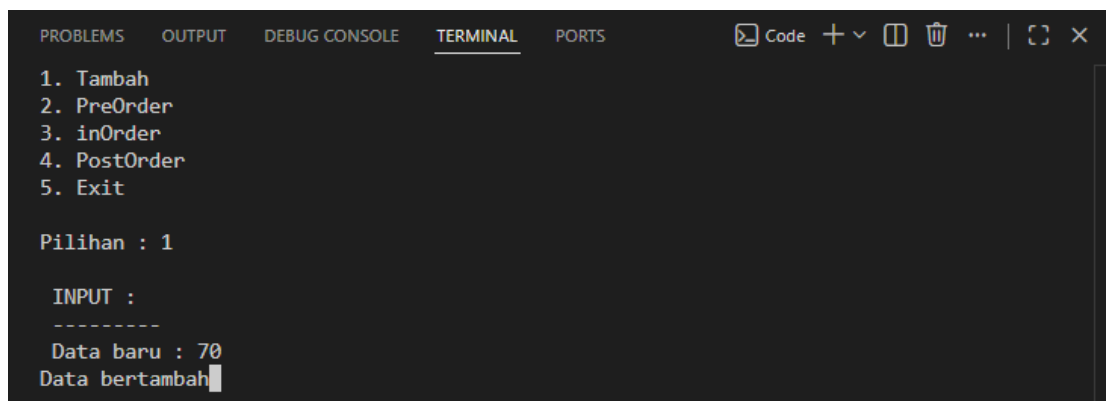


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 30
Data bertambah
```

Gambar 2 Screenshot Output Program Tambah Untuk Input Data

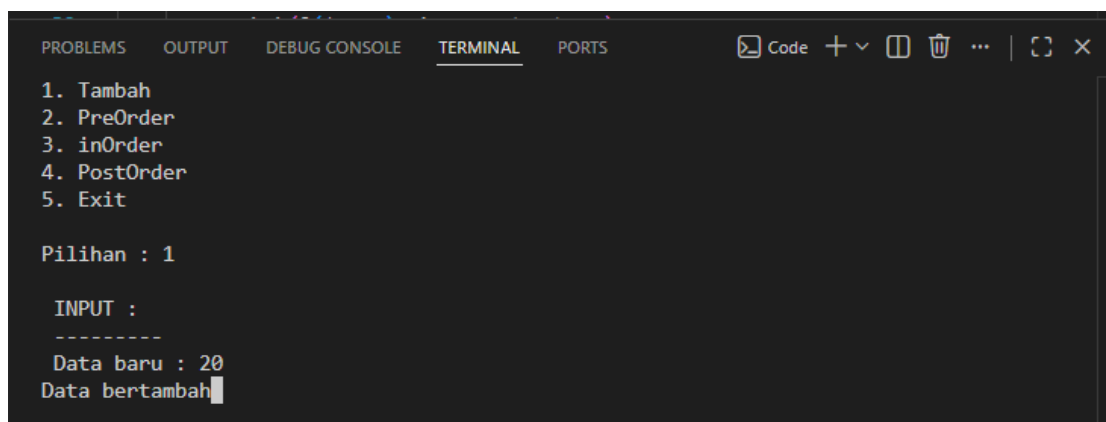


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 70
Data bertambah
```

Gambar 3 Screenshot Output Program Tambah Untuk Input Data

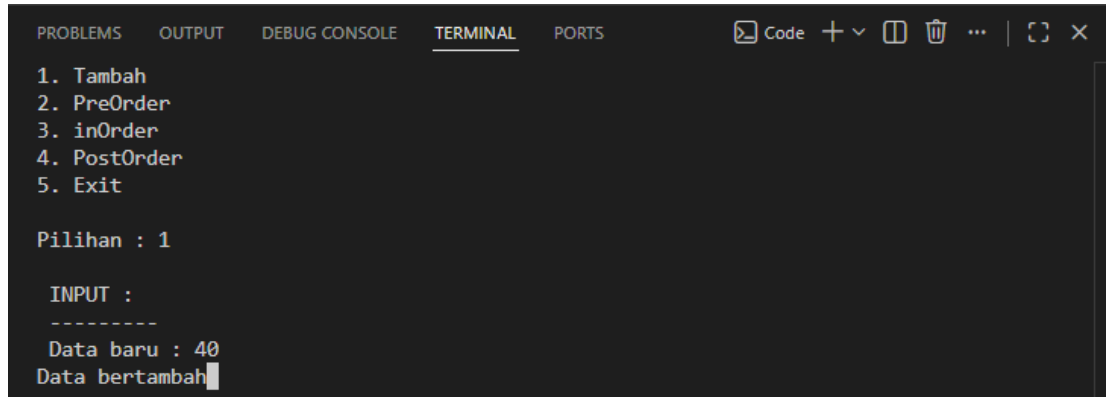


```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 20
Data bertambah
```

Gambar 4 Screenshot Output Program Tambah Untuk Input Data



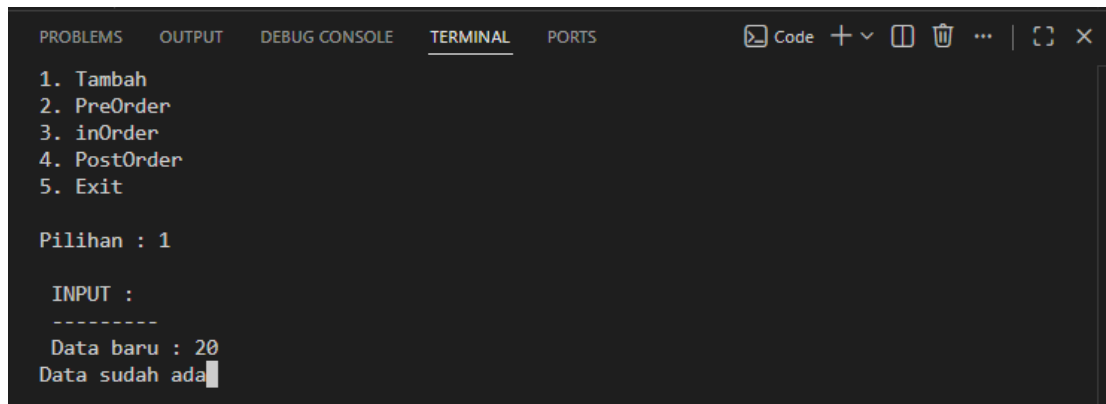
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Code + - [ ] [ ] ... | [ ] [ ] X

1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 40
Data bertambah
```

Gambar 5 Screenshot Output Program Tambah Untuk Input Data



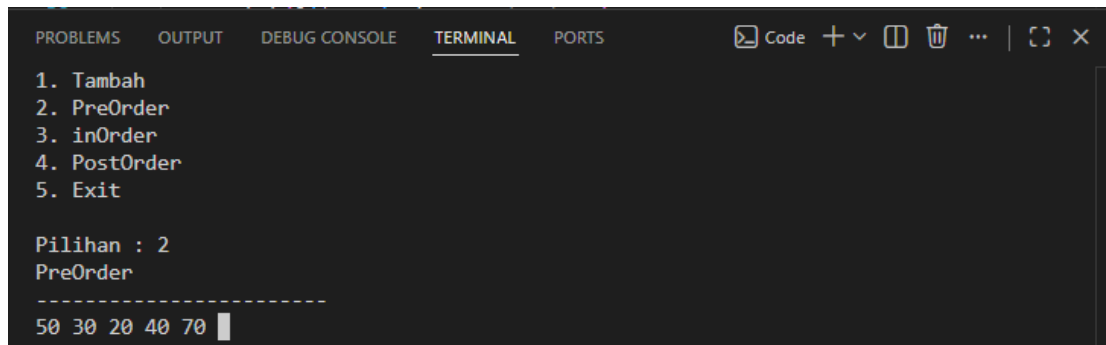
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Code + - [ ] [ ] ... | [ ] [ ] X

1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 1

INPUT :
-----
Data baru : 20
Data sudah ada
```

Gambar 6 Screenshot Output Program Tambah Input Data Yang Sama



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  Code + - [ ] [ ] ... | [ ] [ ] X

1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 2
PreOrder
-----
50 30 20 40 70
```

Gambar 7 Screenshot Output Program PreOrder

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 2
PreOrder
-----
Masih Kosong
```

Gambar 8 Screenshot Output Program PreOrder Data Kosong

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 3
InOrder
-----
20 30 40 50 70
```

Gambar 9 Screenshot Output Program InOrder

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 3
InOrder
-----
Masih Kosong
```

Gambar 10 Screenshot Output Program InOrder Data Kosong

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 4
PostOrder
-----
20 40 30 70 50
```

Gambar 11 Screenshot Output Program PostOrder

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 4
PostOrder
-----
Masih Kosong
```

Gambar 12 Screenshot Output Program PostOrder Data Kosong

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code + - [ ] [ ] ... | [ ] [ ] X
1. Tambah
2. PreOrder
3. inOrder
4. PostOrder
5. Exit

Pilihan : 5
PS C:\Users\Lenovo LOQ\OneDrive\Documents\PRAKTIKUM ALGO_1\Modul_7_Algo>
```

Gambar 13 Screenshot Output Program Exit

C. Pembahasan

- Library dan Deklarasi Struktur Data

Pada baris 1 terdapat `#include <stdio.h>` yang digunakan untuk menyediakan fungsi-fungsi standar bahasa C yang berkaitan dengan operasi input dan output. Pada baris 2 terdapat `#include <conio.h>` yang digunakan untuk menyediakan fungsi `_getch()` sehingga program dapat menunggu pengguna menekan tombol sebelum melanjutkan proses berikutnya. Pada baris 3 terdapat `#include <stdlib.h>` yang digunakan untuk mengakses fungsi sistem seperti `system("cls")` untuk membersihkan layar dan konstanta `EXIT_FAILURE` sebagai nilai pengembalian program. Pada baris 4 terdapat `#include <iostream>` yang digunakan agar program dapat menggunakan objek input dan output seperti `cin` dan `cout`. Pada baris 6 terdapat `using namespace std;` yang digunakan agar elemen-elemen dari namespace standar C++ dapat digunakan tanpa perlu menuliskan awalan `std::`.

Pada baris 8 sampai 12 terdapat deklarasi struktur data `Node` yang digunakan sebagai representasi setiap simpul pada Binary Search Tree (BST). Pada baris 9 terdapat atribut data yang digunakan untuk menyimpan nilai pada node. Pada baris 10 terdapat pointer kiri yang digunakan untuk menunjuk anak kiri dari node saat ini. Pada baris 11 terdapat pointer kanan yang digunakan untuk menunjuk anak kanan dari node saat ini. Struktur ini menjadi dasar pembentukan pohon biner yang akan digunakan pada seluruh proses penyimpanan dan traversal data.

- Function `tambah()` untuk Menambahkan Node ke BST

Pada baris 14 terdapat deklarasi function `tambah(Node **root, int databaru)` yang digunakan untuk menambahkan data baru ke dalam Binary Search Tree. Function ini menerima parameter berupa alamat root pohon dan nilai data yang akan dimasukkan.

Pada baris 15 terdapat kondisi `if (*root == NULL)` yang digunakan untuk memeriksa apakah posisi node yang sedang dituju masih kosong. Kondisi ini merupakan base case dari proses rekursif penambahan node. Ketika posisi kosong ditemukan, maka pada baris 16 sampai 23 program membuat node baru menggunakan

operator new, mengisi atribut data dengan nilai databaru, serta menginisialisasi pointer kiri dan kanan menjadi NULL. Pada baris 21 pointer root diarahkan ke node baru sehingga node tersebut resmi menjadi bagian dari Binary Search Tree. Pada baris 24 program menampilkan pesan "Data bertambah" sebagai informasi bahwa proses penyisipan berhasil dilakukan.

Pada baris 27 terdapat kondisi else if (databaru < (*root)->data) yang digunakan untuk membandingkan nilai data baru dengan data node saat ini. Jika nilai data baru lebih kecil, maka pada baris 28 function tambah() memanggil dirinya sendiri secara rekursif menuju subtree kiri melalui pointer kiri. Pada baris 29 terdapat kondisi else if (databaru > (*root)->data) yang digunakan untuk memeriksa apakah data baru lebih besar dari data node saat ini. Jika kondisi terpenuhi maka pada baris 30 function kembali memanggil dirinya sendiri secara rekursif menuju subtree kanan melalui pointer kanan.

Proses rekursif pada baris 28 dan 30 akan terus berlangsung sampai ditemukan posisi kosong yang sesuai dengan aturan Binary Search Tree. Dengan mekanisme tersebut setiap data yang lebih kecil akan ditempatkan di sisi kiri dan setiap data yang lebih besar akan ditempatkan di sisi kanan.

Pada baris 31 terdapat kondisi else if (databaru == (*root)->data) yang digunakan untuk memeriksa apakah data yang dimasukkan sudah ada di dalam pohon. Jika data sama dengan node yang telah ada, maka pada baris 32 program menampilkan pesan "Data sudah ada" sehingga data duplikat tidak akan ditambahkan ke dalam BST.

- Function preOrder()

Pada baris 35 terdapat deklarasi function preOrder(Node *root) yang digunakan untuk melakukan traversal pohon menggunakan metode PreOrder. Traversal PreOrder memiliki urutan kunjungan Root, Left Subtree, kemudian Right Subtree.

Pada baris 36 terdapat kondisi `if (root != NULL)` yang digunakan untuk memastikan node yang sedang diproses benar-benar ada. Jika node bernilai `NULL`, maka function akan berhenti sehingga rekursi tidak berlanjut ke node yang tidak ada.

Pada baris 37 program menampilkan nilai data dari node saat ini menggunakan `cout << root->data`. Karena perintah ini dijalankan pertama kali, maka node root selalu ditampilkan lebih dahulu. Pada baris 38 function `preOrder(root->kiri)` dipanggil secara rekursif untuk mengunjungi seluruh node pada subtree kiri. Setelah seluruh subtree kiri selesai diproses, pada baris 39 function `preOrder(root->kanan)` dipanggil untuk mengunjungi seluruh node pada subtree kanan. Dengan urutan tersebut traversal akan menghasilkan pola Root–Left–Right.

- Function `inOrder()`

Pada baris 43 terdapat deklarasi function `inOrder(Node *root)` yang digunakan untuk melakukan traversal menggunakan metode InOrder. Traversal InOrder memiliki urutan Left Subtree, Root, kemudian Right Subtree.

Pada baris 44 terdapat kondisi `if (root != NULL)` yang digunakan untuk memastikan node yang diproses masih valid. Pada baris 45 function `inOrder(root->kiri)` dipanggil terlebih dahulu secara rekursif sehingga seluruh subtree kiri diproses sebelum node saat ini ditampilkan.

Setelah seluruh subtree kiri selesai diproses, pada baris 46 program menampilkan data node saat ini menggunakan `cout << root->data`. Selanjutnya pada baris 47 function `inOrder(root->kanan)` dipanggil untuk memproses subtree kanan.

Karena BST selalu menyimpan data yang lebih kecil di kiri dan data yang lebih besar di kanan, maka traversal InOrder akan menghasilkan urutan data dari nilai terkecil hingga terbesar secara otomatis.

- Function `postOrder()`

Pada Pada baris 51 terdapat deklarasi function `postOrder(Node *root)` yang digunakan untuk melakukan traversal menggunakan metode PostOrder. Traversal PostOrder memiliki urutan Left Subtree, Right Subtree, kemudian Root.

Pada baris 52 terdapat kondisi `if (root != NULL)` yang digunakan untuk memastikan node yang diproses masih ada. Pada baris 53 function `postOrder(root->kiri)` dipanggil terlebih dahulu untuk mengunjungi seluruh subtree kiri. Setelah subtree kiri selesai diproses, pada baris 54 function `postOrder(root->kanan)` dipanggil untuk mengunjungi subtree kanan.

Setelah kedua subtree selesai diproses, pada baris 55 program menampilkan data node saat ini menggunakan `cout << root->data`. Karena node root ditampilkan paling akhir, maka traversal PostOrder menghasilkan pola Left-Right-Root dan sering digunakan pada proses penghapusan atau evaluasi struktur pohon

- Function `main()` dan Pengelolaan Menu Program

Pada Pada baris 59 terdapat function `main()` yang merupakan titik awal eksekusi program. Ketika program dijalankan, seluruh proses akan dimulai dari function ini.

Pada baris 60 terdapat deklarasi variabel `pil` yang digunakan untuk menyimpan pilihan menu dari pengguna dan variabel `data` yang digunakan untuk menyimpan nilai yang akan dimasukkan ke dalam Binary Search Tree. Pada baris 61 terdapat deklarasi pointer pohon yang berfungsi sebagai root dari BST. Pada baris 62 pointer tersebut diinisialisasi dengan nilai `NULL` yang menandakan bahwa pohon masih kosong dan belum memiliki node.

Pada baris 64 terdapat struktur perulangan `do-while` yang digunakan agar menu ditampilkan secara berulang. Karena menggunakan `do-while`, menu akan ditampilkan minimal satu kali sebelum kondisi diperiksa.

Pada baris 65 terdapat perintah `system("cls")` yang digunakan untuk membersihkan tampilan layar setiap kali menu akan ditampilkan kembali. Pada baris 66 sampai 70 program menampilkan daftar menu yang terdiri dari Tambah, PreOrder,

InOrder, PostOrder, dan Exit. Pada baris 72 program menerima input pilihan pengguna menggunakan `cin >> pil`.

Pada baris 74 terdapat struktur switch (`pil`) yang digunakan untuk menentukan proses yang akan dijalankan berdasarkan pilihan menu yang dimasukkan pengguna.

Pada baris 75 sampai 80 terdapat case 1 yang digunakan untuk menambahkan data baru ke dalam Binary Search Tree. Program meminta pengguna memasukkan nilai data pada baris 76 sampai 79, kemudian pada baris 80 memanggil function `tambah(&pohon, data)` untuk menyisipkan data ke posisi yang sesuai pada BST.

Pada baris 82 sampai 90 terdapat case 2 yang digunakan untuk menampilkan traversal PreOrder. Program terlebih dahulu memeriksa apakah BST sudah memiliki node melalui kondisi `if (pohon != NULL)` pada baris 85. Jika pohon tidak kosong maka function `preOrder(pohon)` dipanggil pada baris 86. Jika pohon masih kosong maka program menampilkan pesan "Masih Kosong" pada baris 89.

Pada baris 91 sampai 99 terdapat case 3 yang digunakan untuk menampilkan traversal InOrder. Mekanismenya sama dengan case sebelumnya, yaitu memeriksa keberadaan node terlebih dahulu kemudian memanggil function `inOrder(pohon)` apabila pohon tidak kosong.

Pada baris 100 sampai 108 terdapat case 4 yang digunakan untuk menampilkan traversal PostOrder. Program memeriksa apakah BST kosong atau tidak, kemudian menjalankan function `postOrder(pohon)` jika node telah tersedia.

Pada baris 109 sampai 110 terdapat case 5 yang digunakan untuk mengakhiri program. Ketika pengguna memilih menu Exit, program langsung menjalankan `return 0` sehingga function `main()` selesai dan program berhenti tanpa melanjutkan proses berikutnya.

Pada baris 112 terdapat pemanggilan fungsi `_getch()` yang digunakan untuk menahan tampilan program sampai pengguna menekan tombol tertentu sehingga hasil proses masih dapat dilihat sebelum menu ditampilkan kembali.

Pada baris 115 terdapat kondisi `while (pil != 5)` yang digunakan untuk memastikan menu akan terus ditampilkan selama pengguna belum memilih menu Exit.

Ketika nilai pil sama dengan 5, perulangan akan dihentikan dan program selesai dijalankan.

Pada baris 116 terdapat `return EXIT_FAILURE` yang secara teori digunakan untuk mengembalikan status kegagalan kepada sistem operasi. Namun pada program ini baris tersebut tidak pernah dieksekusi karena program sudah terlebih dahulu berhenti melalui `return 0` pada case 5.

LINK GITHUB

<https://github.com/Arthur-369/PRAKTIKUM-Algoritma-dan-Struktur-Data>